

Rapport NFE204

**analyse de la variation des cotations
boursières EURONEXT en fonction de
l'actualité**

NFE204 – 2014/2015

Jamel FILALI

Hicham BEKKALI

Jamel.filali@gmail.com

hichambekkali@gmail.com

1 Présentation du sujet

1.1 INTRODUCTION

Le sujet

L'objectif du projet est de collecter et stocker des données de type documents afin de les étudier et réaliser une analyse prédictive qui nous permettra d'anticiper et prédire si possible les fluctuations des cours de bourse par l'analyse de l'actualité (Actualités des entreprises, réseaux sociaux, commentaires).

Les données

- Constitution d'une base JSON (MONGODB) à partir de données de cotation des sociétés référencées Euronext Paris.
 - Chargement de données historiques (données 2015)
 - Chargement quotidien de données présentées au format JSON (Yahoo API) via un script Python
- Constitution d'une base JSON (MONGODB) à partir des informations de type réseaux sociaux (Tendances twitter, communication entreprise,...)
 - Chargement quotidien de données présentées au format JSON (Twitter API) à partir d'un script Python

Données

Historique des cotations (JSON)

Cours de bourses quotidien au format JSON

Actualités (Flux Rss, réseaux sociaux, revues, journaux) aux formats JSON

Liens :

Cotation

JSON :

https://query.yahooapis.com/v1/public/yql?q=select%20*%20from%20yahoo.finance.historicaldata%20where%20symbol%20%3D%20%22YHOO%22%20and%20startDate%20%3D%20%222009-09-11%22%20and%20endDate%20%3D%20%222010-03-10%22&format=json&env=store%3A%2F%2Fdatatables.org%2Falltables%2Fwithkeys

Réseaux sociaux

JSON : <https://dev.twitter.com/docs/api/1/get/trends/%3Awoeid>

Actualités

XML : <http://feeds.finance.yahoo.com/rss/2.0/headline?s=yhoo®ion=US&lang=en-US>

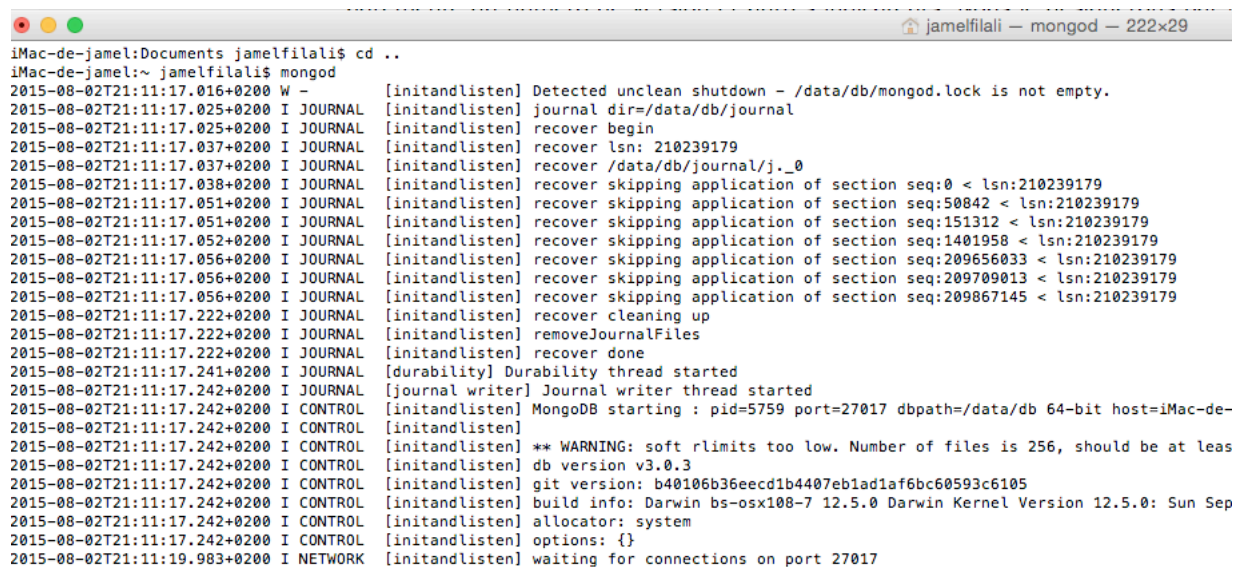
2 Collecte des données boursières

Les étapes

Récupération de mongodb sur le site officiel et définition de variables d'environnement

```
export MONGO_HOME=/usr/local/mongo2.6
export PATH=$PATH:$MONGO_HOME/bin
```

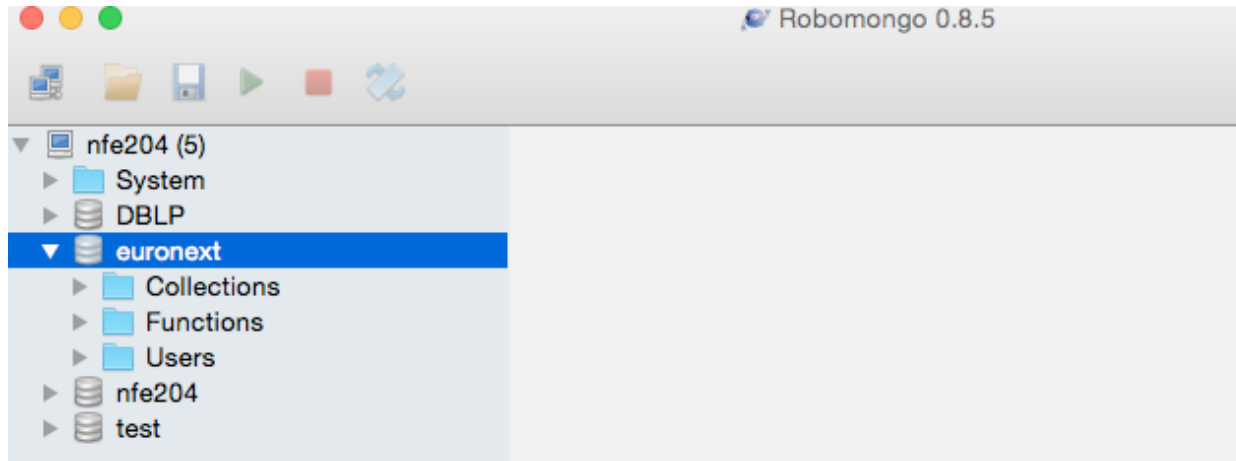
Démarrage mongod



```
iMac-de-jamel:Documents jamelfilali$ cd ..
iMac-de-jamel:~ jamelfilali$ mongod
2015-08-02T21:11:17.016+0200 W - [initandlisten] Detected unclean shutdown - /data/db/mongod.lock is not empty.
2015-08-02T21:11:17.025+0200 I JOURNAL [initandlisten] journal dir=/data/db/journal
2015-08-02T21:11:17.025+0200 I JOURNAL [initandlisten] recover begin
2015-08-02T21:11:17.037+0200 I JOURNAL [initandlisten] recover lsn: 210239179
2015-08-02T21:11:17.037+0200 I JOURNAL [initandlisten] recover /data/db/journal/j._0
2015-08-02T21:11:17.038+0200 I JOURNAL [initandlisten] recover skipping application of section seq:0 < lsn:210239179
2015-08-02T21:11:17.051+0200 I JOURNAL [initandlisten] recover skipping application of section seq:50842 < lsn:210239179
2015-08-02T21:11:17.051+0200 I JOURNAL [initandlisten] recover skipping application of section seq:151312 < lsn:210239179
2015-08-02T21:11:17.052+0200 I JOURNAL [initandlisten] recover skipping application of section seq:1401958 < lsn:210239179
2015-08-02T21:11:17.056+0200 I JOURNAL [initandlisten] recover skipping application of section seq:209656033 < lsn:210239179
2015-08-02T21:11:17.056+0200 I JOURNAL [initandlisten] recover skipping application of section seq:209709013 < lsn:210239179
2015-08-02T21:11:17.056+0200 I JOURNAL [initandlisten] recover skipping application of section seq:209867145 < lsn:210239179
2015-08-02T21:11:17.222+0200 I JOURNAL [initandlisten] recover cleaning up
2015-08-02T21:11:17.222+0200 I JOURNAL [initandlisten] removeJournalFiles
2015-08-02T21:11:17.222+0200 I JOURNAL [initandlisten] recover done
2015-08-02T21:11:17.241+0200 I JOURNAL [durability] Durability thread started
2015-08-02T21:11:17.242+0200 I JOURNAL [journal writer] Journal writer thread started
2015-08-02T21:11:17.242+0200 I CONTROL [initandlisten] MongoDB starting : pid=5759 port=27017 dbpath=/data/db 64-bit host=iMac-de-
2015-08-02T21:11:17.242+0200 I CONTROL [initandlisten] ** WARNING: soft rlimits too low. Number of files is 256, should be at leas
2015-08-02T21:11:17.242+0200 I CONTROL [initandlisten] db version v3.0.3
2015-08-02T21:11:17.242+0200 I CONTROL [initandlisten] git version: b40106b36eecd1b4407eb1ad1af6bc60593c6105
2015-08-02T21:11:17.242+0200 I CONTROL [initandlisten] build info: Darwin bs-osx108-7 12.5.0 Darwin Kernel Version 12.5.0: Sun Sep
2015-08-02T21:11:17.242+0200 I CONTROL [initandlisten] allocator: system
2015-08-02T21:11:17.242+0200 I CONTROL [initandlisten] options: {}
2015-08-02T21:11:19.983+0200 I NETWORK [initandlisten] waiting for connections on port 27017
```

NFE204 – Bases de données documentaires et distribuées

Client Robomongo



Prise en main interface YQL

Création d'une application référencée Yahoo !



Your application has been created successfully!

The Client ID and Client Secret below will allow you to write an application that interacts with [OAuth-enabled](#) Yahoo APIs.

[My Apps](#) / euronextnfe204

Client ID (Consumer Key)

dj0yJmk9dW9yejBkUzVHNkJEJmQ9WVdrOVoyOVRObEppTnpnbWNHbzlNQStJnM9Y29uc3VtZXJzZWNyZXQmeD0xNQ--

Client Secret (Consumer Secret)

0e893b498cff200ca6b46c20569d4cd11c09ba70

You can use Client ID and Client Secret to access Yahoo APIs protected by [OAuth](#).

NFE204 – Bases de données documentaires et distribuées

Récupération des symboles (indices) Euronext Paris sur EODDATA

Exchange:

Euronext Paris

Display

List of Symbols for Euronext Paris [PAR]															DOWNLOAD SYMBOL LIST									
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Z
Code	Name											High	Low	Close	Volume		Change							
AB	AB SCIENCE											15.40	15.05	15.07	35,343		-0.28 ↓ 1.82							
ABCA	ABC ARBITRAGE											4.850	4.820	4.840	11,590		0.000 ▬ 0.00							
ABIO												15.27	15.05	15.07	45,469		-0.23 ↓ 1.50							
AC	ACCOR											44.61	44.01	44.10	978,631		-0.27 ↓ 0.60							
ACA	CREDIT AGRICOLE											13.65	12.72	12.94	35,433,352		-1.47 ↓ 10.21							
ACAN	ACANTHE DEV.											0.4300	0.4200	0.4300	49,140		0.0100 ↑ 2.38							
ADI	AUDIKA GROUPE											17.65	17.55	17.62	1,750		0.02 ↑ 0.11							
ADOC												89.80	86.51	89.23	32,187		0.49 ↑ 0.55							
ADP	ADP											110.3	108.8	109.8	60,267		0.3 ↑ 0.27							
ADVI	ADVINI											31.99	31.10	31.81	63		0.71 ↑ 2.28							
AF	AIR FRANCE -KLM											6.550	6.360	6.380	2,520,938		-0.070 ↓ 1.09							
AFO	AFONE											4.640	4.620	4.620	22		-0.030 ↓ 0.65							
AGTA	AGTA RECORD											47.70	47.70	47.70	2,160		0.00 ▬ 0.00							
AI	AIR LIQUIDE											120.4	118.9	120.3	519,339		0.6 ↑ 0.46							
AIR	Airbus Group											65.77	64.32	64.89	1,579,576		-0.84 ↓ 1.28							
AKA	AKKA TECHNOLOGIES											31.00	30.51	30.52	3,720		0.03 ↑ 0.10							
AKE	ARKEMA											72.95	71.45	72.56	448,833		0.76 ↑ 1.06							
ALA2M	A2MICILE EUROPE											18.55	18.55	18.55	306		0.00 ▬ 0.00							

MEMBER LOGIN

Jamel Filali

[Go To My Account](#)

[Standard Member]

[Log Out](#)

93.26.136.136

France

UPGRADE TO PLATINUM

our most popular subscription level

Every data type and exchange we offer delivered automatically in your format.

STAY ON TOP OF THE MARKET!

Les données symboles sont en entrée de la requête YQL permettant de récupérer l'historique des cotations pour la période du 01/01/2015 au 17/09/2015.

Les données sont stockées dans un fichier plat puis retraitées pour être parcourues par un script python de manière dynamique:

PAR.txt

Symbol	Description
AB.PA	AB SCIENCE
ABCA.PA	ABC ARBITRAGE
ABIO.PA	
AC.PA	ACCOR
ACA.PA	CREDIT AGRICOLE
ACAN.PA	ACANTHE DEV.
ADI.PA	AUDIKA GROUPE
ADOC.PA	
ADP.PA	ADP
ADVI.PA	ADVINI
AF.PA	AIR FRANCE -KLM
AFO.PA	AFONE
AGTA.PA	AGTA RECORD
AI.PA	AIR LIQUIDE
AIR.PA	Airbus Group
AKA.PA	AKKA TECHNOLOGIES
AKE.PA	ARKEMA
ALA2M.PA	A2MICILE EUROPE
ALACI.PA	ACCES INDUSTRIE
ALADA.PA	ADA
ALADM.PA	ADTHINK MEDIA
ALADO.PA	ADOMOS
ALAGR.PA	AGROGENERATION
ALALO.PA	ACHETER-LOUER.FR
ALANT.PA	ANTEVENIO
ALAST.PA	ASTELLIA
ALAUP.PA	AUPLATA
ALBDM.PA	BD MULTI MEDIA
ALBER.PA	STRETRADE

NFE204 – Bases de données documentaires et distribuées

L'objectif est de constituer un fichier JSON avec les données des indices Euronext.
Cette même liste sera chargée dans la base MongoDB.

Test de la requête YQL

Responses

☒ JSON ☐ XML

YQL Query [?](#)

[YQL Console](#)

```
select * from yahoo.finance.historicaldata where symbol = "MMB.PA" and startDate = "2015-01-01" and endDa
```

Test

Response

```
{
  "Symbol": "MMB.PA",
  "Date": "2015-08-03",
  "Open": "27.28",
  "High": "27.54",
  "Low": "27.265",
  "Close": "27.485",
  "Volume": "254800",
  "Adj_Close": "27.485"
},
{
  "Symbol": "MMB.PA"
```

Endpoint

```
https://query.yahooapis.com/v1/public/yql?q=select%20*%20from%20yahoo.finance.historicaldata%20where%20symbol%20'
```

Création de scripts en python

Utilisation de l'IDE PyCHARM CE pour le développement et d'Anaconda comme environnement d'exécution

Key	Value	Type
(1) ObjectId("55dcb7b84121a53c395e406c")	{ 2 fields }	Object
_id	ObjectId("55dcb7b84121a53c395e406c")	ObjectId
query	{ 4 fields }	Object
results	{ 1 fields }	Object
quote	Array [168]	Array
0	{ 8 fields }	Object
Date	2015-08-24	String
Open	23.56	String
Volume	619600	String
Close	23.24	String
Symbol	MMB.PA	String
Low	22.455	String
Adj_Close	23.24	String
High	24.02	String

Il nous faut un format JSON exploitable facilement, nous choisirons la structure suivante :

NFE204 – Bases de données documentaires et distribuées

```
[{
  "_id": "symbol:1",
  "title": "MMB.PA",
  "quote": {
    "date": "2015-08-24"
    "open": "23.56"
    "close": "23.24"
    ...  },
  {"_id": "symbol:2",
    "title": "AF.PA",      ...  } ]
```

Cette structure conserve uniquement les informations essentielles à notre étude à savoir, l'indice à étudier ainsi que les informations concernant la date et les valeurs à l'ouverture et à la clôture des marchés.

Nous pourrions par la suite rajouter un champ « Descriptif » nous permettant d'associer l'indice à son nom connu sur les marchés.

Le format initial récupéré depuis le flux Yahoo est de la forme suivante :

```
{ "query": { "count": 186, "created": "2015-09-20T16:26:27Z", "lang": "fr-FR", "results": { "quote": [ { "Symbol": "MMB.PA", "Date": "2015-09-17", "Open": "25.775", "High": "25.82", "Low": "25.545", "Close": "25.755", "Volume": "180000", "Adj_Close": "25.755" }, { "Symbol": "MMB.PA", "Date": "2015-09-16", "Open": "25.615", "High": "25.835", "Low": "25.51", "Close": "25.67", "Volume": "277000", "Adj_Close": "25.67" }, { "Symbol": "MMB.PA", "Date": "2015-09-15", "Open": "25.34", "High": "25.53", "Low": "25.145", "Close": "25.445", "Volume": "361200", "Adj_Close": "25.445" }, { "Symbol": "MMB.PA", "Date": "2015-09-14", "Open": "25.40", "High": "25.61", "Low": "25.12", "Close": "25.265", "Volume": "389100", "Adj_Close": "25.265" }, { "Symbol": "MMB.PA", "Date": "2015-09-11", "Open": "25.77", "High": "25.77", "Low": "25.245", "Close": "25.38", "Volume": "289400", "Adj_Close": "25.38" }, { "Symbol": "MMB.PA", "Date": "2015-09-10", "Open": "25.775", "High": "26.14", "Low": "25.505", "Close": "25.735", "Volume": "424500", "Adj_Close": "25.735" } ] }
```

La liste d'information étant trop exhaustive, nous avons fait le choix de réduire cette liste d'information.

NFE204 – Bases de données documentaires et distribuées

Script de récupération de l'historique

```
__author__ = 'jamelfilali'
import urllib.request, urllib, json as js, pymongo, time # On récupère les librairies nécessaires

def histo():

    try:
        f = open('PAR.txt', 'rU') # On charge le fichier des symboles Euronext

        for line in f.readlines(): # On boucle sur les symboles du fichiers

            line=line[:-1]

            # Ici on fait appel au webservice fourni par Yahoo pour l'historique des cotations
            baseurl =
            "https://query.yahooapis.com/v1/public/yql?q=select%20*%20from%20yahoo.finance.historicaldata%20where%20symbol%20%3D%20%27"+line+"%27%20and%20startDate%20%3D%20%272015-01-01%27%20and%20endDate%20%3D%20%272015-09-17%27&format=json&env=store%3A%2F%2Fdatatables.org%2Falltableswithkeys"

            result = urllib.request.urlopen(baseurl).read()

            data = js.loads(result.decode())

            import json
            with open('data.txt', 'w') as outfile:
                json.dump(data, outfile) #Le Json reçu est stocké dans un fichier texte

            MONGODB_URI = 'mongodb://localhost:27017/euronext' # Adresse de la base Mongo

            client = pymongo.MongoClient(MONGODB_URI)

            db = client.get_default_database()

            stocks = db['stocks'] # On crée une base 'stocks' si elle n'existe pas

            stocksDict = {} # On crée un dictionnaire Python

            children = [] # On crée un array pour manipuler les données reçues et les reformater comme bon nous
            semble

            if data['query']['results'] != None:
                for vl in data['query']['results']['quote']:
                    stocksDict["Title"] = vl['Symbol']
                    children.append(({ "date" : vl['Date'], "open" : vl["Open"], "close" : vl['Close']}))

            stocksDict["Quote"] = children # On insère l'array avec les données reformattées dans le dictionnaire
            Json

            stocks.insert(stocksDict) # On insère le Json en base

            client.close()

            time.sleep(1) # Temporisation

    except TypeError: # Gestion optionnelle des cas d'erreur
```

NFE204 – Bases de données documentaires et distribuées

raise

histo() # commande principale permettant de lancer la fonction histo()

Vue Robomongo du résultat 'recupHistorique.py'

- Chaque symbole (indice) est associé à un identifiant Objectid généré automatiquement à l'insertion

stocks2 0 sec.			0
Key	Value	Type	
(1) Objectid("55f4a8004121a54bfc0e6ccf")	{ 3 fields }	Object	
_id	Objectid("55f4a8004121a54bfc0e6ccf")	Objectid	
Quote	Array [182]	Array	
Title	MMB.PA	String	

(1) Objectid("55f593ac4121a55b0cedeeaf")	{ 3 fields }	Object	
_id	Objectid("55f593ac4121a55b0cedeeaf")	Objectid	
Title	AB.PA	String	
Quote	Array [182]	Array	
(2) Objectid("55f593ad4121a55b0cedeeb0")	{ 3 fields }	Object	
_id	Objectid("55f593ad4121a55b0cedeeb0")	Objectid	
Title	ABCA.PA	String	
Quote	Array [182]	Array	
(3) Objectid("55f593ae4121a55b0cedeeb1")	{ 3 fields }	Object	
_id	Objectid("55f593ae4121a55b0cedeeb1")	Objectid	
Title	ABIO.PA	String	
Quote	Array [182]	Array	
(4) Objectid("55f593b24121a55b0cedeeb2")	{ 3 fields }	Object	
_id	Objectid("55f593b24121a55b0cedeeb2")	Objectid	
Title	AC.PA	String	

- Les informations sont récupérées pour chaque journée de cotation et sauvegardées

(1) Objectid("55f4a8004121a54bfc0e6ccf")	{ 3 fields }	Object	
_id	Objectid("55f4a8004121a54bfc0e6ccf")	Objectid	
Quote	Array [182]	Array	
0	{ 3 fields }	Object	
1	{ 3 fields }	Object	
date	2015-09-10	String	
close	25.735	String	
open	25.775	String	
2	{ 3 fields }	Object	
date	2015-09-09	String	
close	25.875	String	
open	26.00	String	

NFE204 – Bases de données documentaires et distribuées

Script de récupération quotidienne des cotations

```
import urllib.request,json,pymongo, datetime # On récupère les librairies nécessaires
import schedule # Librairie utile pour programmer un job quotidien
import time

data = {} # On crée un dictionnaire Python

def get_daily_quote(line): #Programme pour l'appel webservice de Yahoo permettant la récupération quotidienne
des cotations Euronext
    url_to_open =
    "https://query.yahooapis.com/v1/public/yql?q=select%20*%20from%20yahoo.finance.quotes%20where%20symbol%20in%20(%22"+line+"%22)&format=json&diagnostics=true&env=store%3A%2F%2Fdatatables.org%2Falltableswithkeys&callback="
    result = urllib.request.urlopen(url_to_open).read()
    return result

def job(): # Job quotidien
    f = open('PAR.txt', 'rU')

    for line in f.readlines():

        line=line[:-1]

        data = get_daily_quote(line) # Appel au webservice avec symboles en paramètres

        data = json.loads(data.decode())

        stocksDict = {}

        children = []

        vl = data['query']['results']['quote']

        stocksDict["Title"] = vl.get("Symbol", None)

        children.append(({ "open" : vl.get("Open", None), "close" : vl.get("LastTradePriceOnly", None), "date" :
datetime.datetime.strptime(vl.get("LastTradeDate", None), "%m/%d/%Y").strftime('%Y-%m-%d')}))

        stocksDict["Quote"] = children

        MONGODB_URI = 'mongodb://localhost:27017/euronext'

        client = pymongo.MongoClient(MONGODB_URI)

        db = client.get_default_database()

        stocks = db['stocks']

        stocks.update({'Title':stocksDict["Title"]},{ '$push': { "Quote" : stocksDict["Quote"]}}) # Fonction
update dans mongoDB qui insère les données à la suite, Le critère 'Title' = Symbole permet d'insérer au bon
endroit
```

NFE204 – Bases de données documentaires et distribuées

```
client.close()

schedule.every().day.at("20:00").do(job) # On programme le job tous les jours à 20:00
```

```
while True:
    schedule.run_pending()
    time.sleep(1)
```

Explications

- Un webservice permet de récupérer les informations quotidiennes est exposé par Yahoo
- L'idée est de parcourir le fichier des symboles quotidiennement grâce à l'utilisation de la librairie Python 'Schedule' et d'exécuter le script à une heure définie, nous choisirons 20h30 (Clôture des marchés à 17h30)
- Le résultat JSON de la requête est sensiblement différent de la requête historique :
 - ```
results":{"quote":{"symbol":"MMB.PA","Ask":"25.51","AverageDailyVolume":"396894","Bid":"25.49","AskRealtime":null,"BidRealtime":null,"BookValue":"15.14","Change_PercentChange":"-0.24 - 0.93%","Change":"-0.24","Commission":null,"Currency":"EUR","ChangeRealtime":null,"AfterHoursChangeRealtime":null,"DividendShare":null,"LastTradeDate":"9/18/2015","TradeDate":null,"EarningsShare":"0.65","ErrorIndicationreturnedforsymbolchangedinvalid":null,"EPSEstimateCurrentYear":"2.92","EPSEstimateNextYear":null,"EPSEstimateNextQuarter":"0.00","DaysLow":"24.99","DaysHigh":"25.67","YearLow":"17.83","YearHigh":"30.23","HoldingsGainPercent":null,"AnnualizedGain":null,"HoldingsGain":null,"HoldingsGainPercentRealtime":null,"HoldingsGainRealtime":null,"MoreInfo":null,"OrderBookRealtime":null,"MarketCapitalization":"3.27B","MarketCapRealtime":null,"EBITDA":"532.00M","ChangeFromYearLow":"7.68","PercentChangeFromYearLow":"+43.10%","LastTradeRealtimeWithTime":null,"ChangePercentRealtime":null,"ChangeFromYearHigh":"-4.71","PercebtChangeFromYearHigh":"-15.58%","LastTradeWithTime":"5:35pm - 25.51","LastTradePriceOnly":"25.51","HighLimit":null,"LowLimit":null,"DaysRange":"24.99 - 25.67","DaysRangeRealtime":null,"FiftydayMovingAverage":"25.51","TwoHundreddayMovingAverage":"26.89","ChangeFromTwoHundreddayMovingAverage":"-1.37","PercentChangeFromTwoHundreddayMovingAverage":"-5.10%","ChangeFromFiftydayMovingAverage":"0.00","PercentChangeFromFiftydayMovingAverage":"+0.01%","Name":"LAGARDERE SCA N","Notes":null,"Open":"25.67","PreviousClose":"25.51","PricePaid":null,"ChangeinPercent":"-0.93%","PriceSales":"0.44","PriceBook":"1.69","ExDividendDate":"5/8/2015","PERatio":"39.19","DividendPayDate":null,"PERatioRealtime":null,"PEGRatio":"0.00","PriceEPSEstimateCurrentYear":null,"PriceEPSEstimateNextYear":null,"Symbol":"MMB.PA","SharesOwned":null,"ShortRatio":"0.00","LastTradeTime":"5:35pm","TickerTrend":null,"OneyrTargetPrice":null,"Volume":"540016","HoldingsValue":null,"HoldingsValueRealtime":null,"YearRange":"17.83 - 30.23","DaysValueChange":null,"DaysValueChangeRealtime":null,"StockExchange":"PAR","DividendYield":null,"PercentChange":"-0.93%"}}}
```

## NFE204 – Bases de données documentaires et distribuées

On le voit ici, le JSON est beaucoup plus verbeux. Les informations pertinentes à récupérer sont à retrouver dans les balises Open, LastTradePriceOnly et LastTradeDate. Un travail de reformatage de la date est nécessaire pour coller au format de la date issu de la requête historique.

- Autre point de difficulté, il faut insérer les données dans le bon format en insérant au « bon » endroit. A la manière d'une requête SQL nous permettant de faire un Update dans une table selon un critère défini « UPDATE « Table » SET « champ » = « valeur » WHERE Key = « cle ». La librairie Pymongo permet de faire cela et nous préserve de l'écrasement des données existantes.

```
stocks.update({'Title':stocksDict["Title"]},{'$push': { "Quote" : stocksDict["Quote"]}})
```

La key est ici matérialisée par la valeur stocksDict["Title"].

Le mot clé **\$push** nous permet d'insérer les données sans écraser les existantes.

- Quid du rollback dans MongoDB : A lire la documentation officielle de MongoDB (<http://docs.mongodb.org/manual/tutorial/perform-two-phase-commits/>), une façon particulière de mise à jour des données appelée le two phase commits permet de créer un mécanisme de rollback semblable au mécanisme si précieux dans le monde relationnel. J'ai d'ailleurs été confronté à la panne de type « connexion réseau KO » : Que faire dans ces moments là, surtout lorsqu'une partie de la base a été updatée et que l'autre reste en attente de mise à jour. Nous pourrions étendre l'étude par la mise en place de ReplicaSet nous assurant un mécanisme de reprise sur panne.

## **3 Collecte des données Twitter**

### **Introduction**

Comme décrit dans l'introduction générale, le projet StockmarketPerdictor permet de consolider les informations de cotations boursières récupérées depuis Yahoo finance, et les twitts des sociétés concernées.

Le marché choisi pour le moment est EURONEXT. Mais il n'est pas impossible de se tourner vers le marché US, NASDAQ ou NYSE. En effet, les twitts relatifs aux sociétés de ces marchés, sont abondantes et bien structurés.

A titre d'exemple, pour chercher les twitts concernant la société APPLE dont le symbol est AAPL, il suffit d'utiliser le mot clef « \$APPL ».

Ce changement de cible n'aura aucun impact sur le code de récupération des twitts, seul le référentiel des sociétés sera modifié.

### **Les outils utilisés**

- Python et les packages twitter, json, pymongo, pandas.
- Pycharm l'IDE python
- MongoDB et Robomongo
- Twitter API

### **La démarche**

Pour pouvoir interagir avec twitter, il faut créer une application et obtenir les informations d'accès.

## NFE204 – Bases de données documentaires et distribuées

**Applications**  
These are the apps that can access your Twitter account. [Learn more](#)

**Facebook Connect**  
Post Tweets to your Facebook profile or page. [Having trouble? Learn more.](#)

**StockmarketPredictor** by [application using Maching learning algorithm against twitt](#)  
to predict the overall stock market trends  
Permissions: read-only

### Récupération des jetons d'accès depuis twitter

#### StockmarketPredictor

[Details](#) [Settings](#) [Keys and Access Tokens](#) [Permissions](#)

 application using Maching learning algorithm against twitts to predict the overall stock market trends  
<http://www.stockmarketPredictor.com>

**Organization**  
*Information about the organization or company associated with your application. This information is optional.*

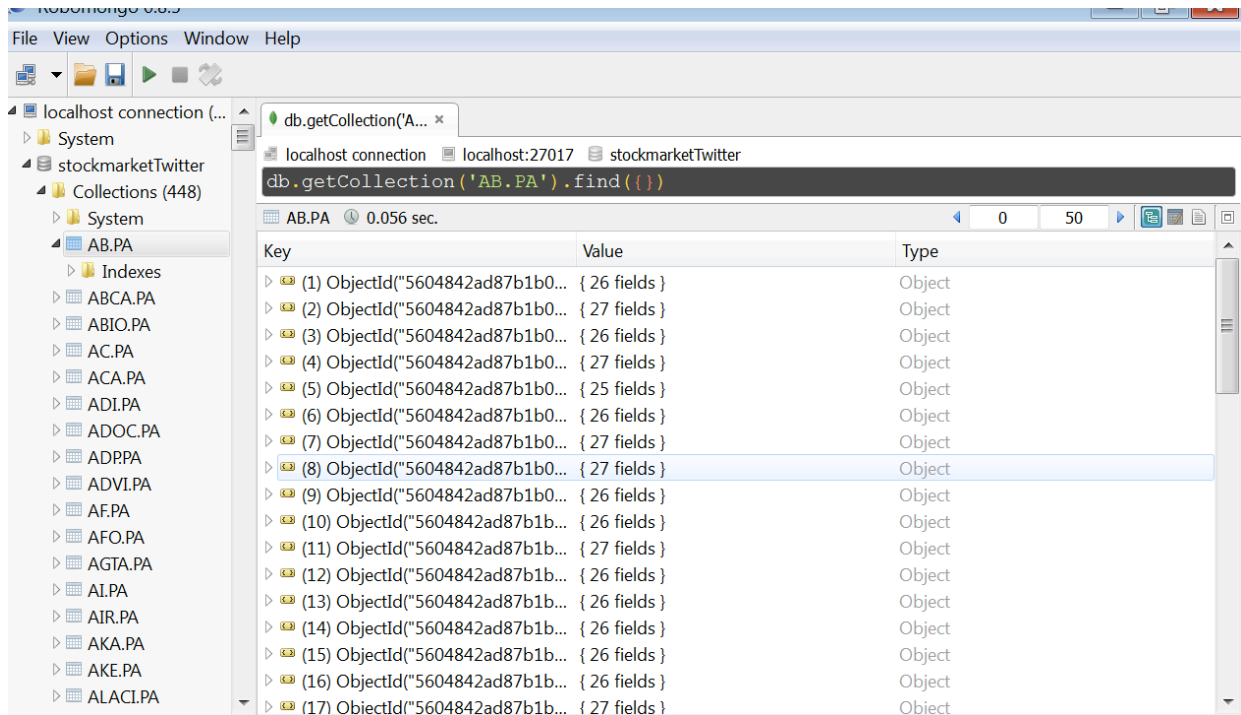
|                      |      |
|----------------------|------|
| Organization         | None |
| Organization website | None |

**Application Settings**  
*Your application's Consumer Key and Secret are used to [authenticate](#) requests to the Twitter Platform.*

|                         |                                                                                                       |
|-------------------------|-------------------------------------------------------------------------------------------------------|
| Access level            | Read-only ( <a href="#">modify app permissions</a> )                                                  |
| Consumer Key (API Key)  | j7XtdgKULvDyVpYB45X31cZF7 ( <a href="#">manage keys and access tokens</a> )                           |
| Callback URL            | None                                                                                                  |
| Callback URL Locked     | No                                                                                                    |
| Sign in with Twitter    | Yes                                                                                                   |
| App-only authentication | <a href="https://api.twitter.com/oauth2/token">https://api.twitter.com/oauth2/token</a>               |
| Request token URL       | <a href="https://api.twitter.com/oauth/request_token">https://api.twitter.com/oauth/request_token</a> |
| Authorize URL           | <a href="https://api.twitter.com/oauth/authorize">https://api.twitter.com/oauth/authorize</a>         |
| Access token URL        | <a href="https://api.twitter.com/oauth/access_token">https://api.twitter.com/oauth/access_token</a>   |

# NFE204 – Bases de données documentaires et distribuées

## Création d'une base de données mongoDB : stockmarketTwitter



## Scripts Python de récupération des informations

```
__author__ = 'root'

import twitter

import pymongo

import json

import pandas as pd

def oauth_login():

 CONSUMER_KEY = 'xxxxxxxxxxxxxxxxxxxxxx'

 CONSUMER_SECRET = 'xxxxxxxxxxxxxxxxxxxxxxxxxx'
```

```
OAUTH_TOKEN = 'xxxxxxxxxxxxxxxxxxxxxxxx'

OAUTH_TOKEN_SECRET = 'xxxxxxxxxxxxxxxxxxxxxxxx'

 auth = twitter.oauth.OAuth(OAUTH_TOKEN,
OAUTH_TOKEN_SECRET,CONSUMER_KEY, CONSUMER_SECRET)

 twitter_api = twitter.Twitter(auth=auth)

 return twitter_api

def twitter_search(twitter_api, q, max_results=1000, **kw):

 search_results = twitter_api.search.tweets(q=q, count=1000,
**kw)

 statuses = search_results['statuses']

 # Enforce a reasonable limit
 max_results = min(1000, max_results)

 for _ in range(10): # 10*100 = 1000
 try:

 next_results =
search_results['search_metadata']['next_results']

 except KeyError:

 break
```



```
 kwargs = dict([kv.split('=')
 for kv in next_results[1:].split("&")])

 search_results = twitter_api.search.tweets(**kwargs)
 statuses += search_results['statuses']

 if len(statuses) > max_results:
 break

 return statuses

def save_to_mongo(data, mongo_db, mongo_db_coll,
**mongo_conn_kw):

 client = pymongo.MongoClient(**mongo_conn_kw)
 db = client[mongo_db]
 coll = db[mongo_db_coll]

 # Perform a bulk insert and return the IDs
 return coll.insert(data)

#####

twitter_api = oauth_login()

symbolListFile = "data/euronext_Symbols.csv"
symbolList = pd.read_csv(symbolListFile, sep=';', dtype=str)
```

```
symb = symbolList["Symbol"]
lib = symbolList["Libelle"]
n = symbolList.shape[0]

for i in range(n):
 print(i)
 symbol = symb.iloc[i]
 libelle = lib.iloc[i]
 if (isinstance(libelle, float)): libelle = symbol
 if (libelle is None): libelle = symbol
 libelle= libelle.replace (" ", "%20")

 q = libelle
 results = twitter_search(twitter_api, q, max_results=1000)
 if (len(results)!=0):
 save_to_mongo(results, 'stockmarketTwitter', symbol)
```

### Export Mongoddb

Suite à la récupération des différents twitts et stockage dans la base mongoddb, nous pouvons faire un export de toute la base de donnees stockmarketTwitter, en utilisant la commande suivante:

```
mongodump
/d stockmarketTwitter
/o C:\Users\root\...\002_Data\mongoddbstockmarketTwitter.dump
```

## **4 Analyse comparative : Stockage des séries temporelles – MongoDB versus Cassandra**

### **4.1 INTRODUCTION**

Les séries temporelles se retrouvent dans de multiples typologies de données, nous citerons par exemple les données issues :

- Des marchés financiers
- De capteurs (Température, Pression)
- Des réseaux sociaux (Mise à jour de statuts)
- De données de réseaux Mobile

Ce type de données se retrouve la plupart du temps dans de grands gisements de données ordonnés dans le temps et potentiellement agrégés pour faciliter leur lecture.

Dans l'optique d'améliorer la prédiction de certains types d'évènements, il est parfois utile d'avoir à disposition des valorisations de données « timestampées ».

Les bases de données classiques gèrent bien la notion de timestamp : un champ « date » disposé dans une table avec des événements disposés en ligne suffit généralement à répondre l'exigence.

Qu'en est-il des bases de données documentaires ?

De la même façon, nous pouvons stocker un document timestampé par jour. Dans la base de cotations constituées, un exemple pourrait être le suivant :

```
"_id" : ObjectId("560060d54121a5576ce0e6a4"),
"Title" : "EPCP.PA",
"Quote" : [{ "date" : "2015-05-22",
 "open" : "375.95",
 "close" : "375.95" }]
```

De manière générale, les documents peuvent être stockés en prenant en compte les événements ou bien une notion de temps (jours, minutes, secondes). La typologie de données induit la temporalité du stockage.

Le document stocké doit être modélisé de telle sorte à pouvoir être écrit et lu aisément.

## **4.2 STOCKAGE ET LECTURE DES DOCUMENTS DANS MONGODB**

Comme nous l'avons vu, la typologie de données induit une « façon de stocker » les données. Voici les types de documents ainsi que leurs caractéristiques dans la vision MongoDB :

### Document par évènement

```
{
 server: "server1",
 load: 92,
 ts: ISODate("2013-10-16T22:07:38.000-0500")
}
```

Approche de type BDD relationnel

Schéma préférentiel pour l'insertion de documents sans mise à jour nécessaire.

Les données sont généralement agrégées au niveau de l'application émettrice

### Document avec données à la seconde (Pour 1 minute de traitement)

```
{ segId: "l80_mile23",
 speed: { 0: 63, 1: 58, ..., 58: 66, 59: 64 }
 ts: ISODate("2013-10-16T22:07:00.000-0500")
}
```

Les données sont généralement agrégées par minute au niveau de l'application émettrice

Approche de type update, pour 1 écriture en base, 59 mises à jour nécessaires

Granularité fine

### Document avec données à la seconde (Pour 1h de traitement)

```
{
 server: "server1",
 load: {
 0: {0: 15, ..., 59: 45},

 59: {0: 25, ..., 59: 75}
 ts: ISODate("2013-10-16T22:00:00.000-0500")
}
```

Stockage toutes les secondes des données pour l'heure passée

Il faut à chaque seconde mettre à jour les données déjà en base

Pour 1 écriture en base, 3599 update requis

Par exemple, l'approche par évènement implique plus d'écritures que de mises à jour comparé aux autres modèles, MongoDB préconise le mode Update qui est moins consommateur.

De la même façon, en mode lecture, l'approche par évènement est déconseillée. Par exemple, pour des documents générés à la seconde, charger 1h de traitement requiert 3600 lectures en base de document par évènement contre seulement 60 lectures dans l'approche 1 document par minute (granularité à la seconde).

**Les caractéristiques des documents influent sur les performances en termes de lecture de collections.** En effet, si l'on prend l'exemple de données générées toutes les secondes.

Les documents insérés en base selon leur temporalité n'impliqueront pas le même nombre d'accès en lecture.

En 1 heure, pour un document généré par événement (l'événement associé est la création de la donnée à la seconde, il faut 3600 lectures pour reconstituer la totalité des données

A contrario, pour un document généré à la minute, seules 60 lectures sont nécessaires. Des considérations de performance sont donc à prendre en compte.

### **4.3 SÉRIES TEMPORELLES DANS MONGODB**

MongoDB apporte ses préconisations dans l'élaboration d'une base de données documentaires :

- MongoDB privilégie une architecture scale-out permettant d'ajuster le nombre de nœuds en fonction de la volumétrie à prévoir, qui peut être exponentielle dans le cas des séries temporelles. 2 visions à adopter :
  - Une vision verticale : On doit pouvoir monter en puissance sur les machines (CPU, mémoire). Les machines doivent être extensibles.
  - Une vision horizontale : Les données sont partitionnées et la charge de travail correctement répartie.
- En fonction de l'usage prévue des données, il est important de choisir le bon modèle de données pour assurer de bonnes performances en lecture/écriture des données
- De plus il faut également s'interroger sur les connecteurs « analytiques » de type Hadoop, ou bien les langages de requêtage. En effet, il est important de bien comprendre les besoins analytiques pour identifier le meilleur design de la base ainsi qu'un accès le plus optimisé possible.

### **4.4 CASSANDRA: UN OUTIL NOSQL**

Cassandra est souvent décrit comme étant l'outil NoSQL « Killer » dans le cas des séries temporelles. C'est un SGBD NoSQL porté par la fondation Apache répondant au mieux aux problématiques Big Data grâce à sa scalabilité horizontale.

Principales caractéristiques (repris du site developpez.com)

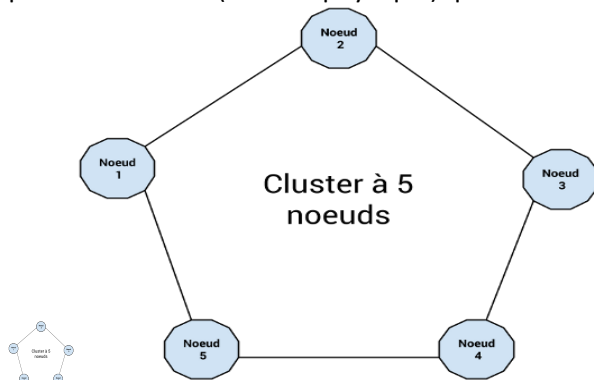
- **Tolérance aux pannes** : Les données d'un nœud (un nœud est une instance de Cassandra) sont automatiquement répliquées vers d'autres nœuds (différentes machines). Ainsi, si un nœud est hors service les données présentes sont disponibles à travers d'autres nœuds. Le terme de facteur de réplication désigne le nombre de nœuds où la donnée est répliquée. Par ailleurs, l'architecture de Cassandra définit le terme de cluster comme étant un groupe d'au moins deux nœuds et un data center comme étant des clusters délocalisés. Cassandra permet d'assurer la réplication à travers différents data center. Les nœuds qui sont tombés peuvent être remplacés sans indisponibilité du service.

# NFE204 – Bases de données documentaires et distribuées

- **Décentralisé** : dans un cluster tous les nœuds sont égaux. Il n'y pas de notion de maitre, ni d'esclave, ni de processus qui aurait à sa charge la gestion, ni même de goulet d'étranglement au niveau de la partie réseau. Nous étudierons dans un prochain article lié à la scalabilité le protocole GOSSIP utilisé pour découvrir la localisation et les informations sur l'état des nœuds d'un cluster.
- **Modèle de données riche** : Le modèle de données proposé par Cassandra basé sur la notion de clé/valeur permet de développer de nombreux cas d'utilisation dans le monde du Web.
- **Élastique** : La scalabilité est linéaire. Le débit d'écriture et de lecture augmente de façon linéaire lorsqu'un nouveau serveur est ajouté dans le cluster. Par ailleurs, Cassandra assure qu'il n'y aura pas d'indisponibilité du système ni d'interruption au niveau des applications.
- **Haute disponibilité** : Possibilité de spécifier le niveau de cohérence concernant la lecture et l'écriture. On parle alors de Tuneable Consistency. Apache Cassandra ne dispose pas de transaction. L'écriture des données est très rapide comparée au monde des bases de données relationnelles.

## Principes de stockage

Quand on parle de Cassandra, on parle souvent de cluster. Un cluster est un regroupement de plusieurs nœuds (serveur physique) qui communiquent entre eux pour la gestion de données.



Les nœuds communiquent entre eux par un protocole peer-to-peer qu'on appelle le Gossip (bavardage).

### *Le théorème de CAP*

- C pour Consistency)
- La disponibilité (A pour Availability)
- La tolérance aux pannes et aux coupures réseaux (P pour Partition-tolerance)

Le théorème postule que pour toute base de données distribuée, on ne peut choisir que 2 de ces 3 paramètres, jamais les 3 en même temps.

Cassandra fait clairement le choix de AP pour une tolérance aux pannes et une disponibilité absolue. En contrepartie, Cassandra sacrifie la cohérence absolue (au sens ACID du terme) contre une cohérence finale, c'est à dire une cohérence forte obtenue après une convergence des données (garantie par un ensemble de mécanisme d'anti-entropie)

# NFE204 – Bases de données documentaires et distribuées

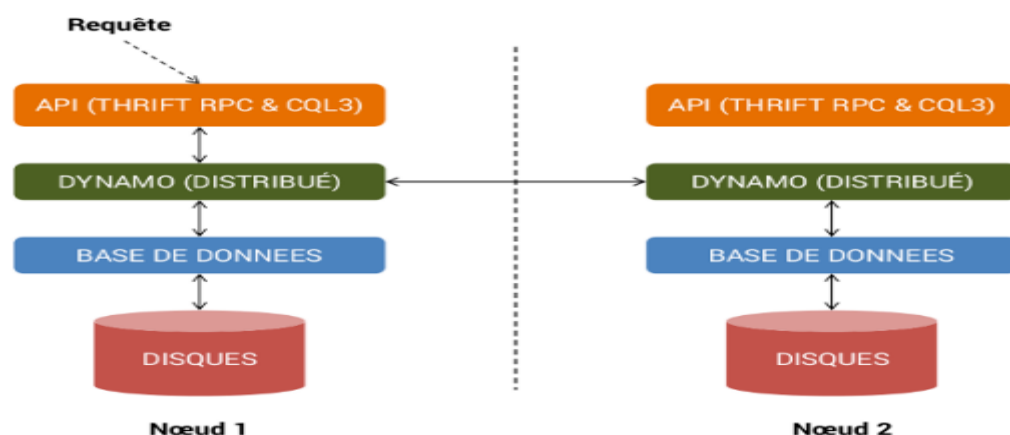
## Architecture

L'architecture Cassandra s'inspire énormément du papier de recherche Big Table de Google , ainsi que de l'architecture Dynamo d'Amazon. Le moteur de stockage de C\* dérive directement de Big Table alors que sa couche de distribution de données s'inspire de l'architecture de Dynamo.

On distingue la présence de 3 couches métiers :

- API, responsable de recevoir les requêtes venant des clients sous format Thrift (protocole RPC) ou dans le nouveau format binaire CQL3
- Dynamo, responsable de la distribution des données entre différents noeuds et du protocole peer-to-peer
- Base de données, responsable de la persistance des données sur disques.

On s'intéressera uniquement à la couche base de données.

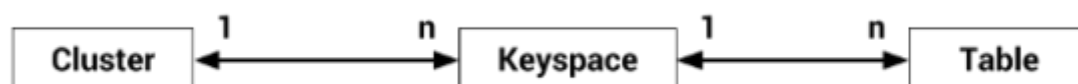


## Modèle physique de données

### KEYSPACE ET TABLES

Dans un cluster Cassandra, on trouve des tables et des keyspaces. Un keyspace peut-être vu comme une base. En effet, dans un modèle multi-tenant on sépare les données de chaque partie dans des keyspaces.

A l'intérieur de chaque keyspace, on trouve des tables. Une table dans C\* a la même signification qu'une table dans le monde SQL, avec des lignes et des colonnes.



| #partition | Colonne  | Colonne  | Colonne  | Colonne | Colonne | ... |
|------------|----------|----------|----------|---------|---------|-----|
| Partition1 | #col1    | #col2    | #col3    |         |         |     |
|            | cellule1 | cellule2 | cellule3 |         |         |     |

## NFE204 – Bases de données documentaires et distribuées

| #partition | Colonne  | Colonne | Colonne  | Colonne | Colonne  | ... |
|------------|----------|---------|----------|---------|----------|-----|
| Partition2 | #col1    | #col2   |          |         |          |     |
|            | cellule1 |         |          |         |          |     |
| Partition3 | #col1    | #col2   | #col3    | #col4   | #col5    |     |
|            | cellule1 |         | cellule3 |         | cellule5 |     |
| Partition4 | #col1    |         |          |         |          |     |
|            | cellule1 |         |          |         |          |     |

Les données sont disposées sur des lignes (qu'on appelle partitions), et au sein de chaque partition on trouve une série de colonnes avec #col/cellule qu'on peut assimiler à une série de clé/valeur.

Pour résumer, une table peut être visualisée conceptuellement comme un ensemble de :

<Clé de partition, <Clé de colonne, cellule>>

A noter que les colonnes sont triées par leur clé de colonne. Nous verrons par la suite que cette fonctionnalité est cruciale pour la modélisation de données dans Cassandra.

### STRUCTURE D'UNE TABLE

Maintenant, nous allons voir comment une table est représentée dans le moteur de stockage (storage engine) de C\*.

Il faut savoir d'abord que les tables sont très fortement typées. Par type, on entend le type de :

1. la clé de partition (#partition)
2. la clé de colonne (#col)
3. la cellule

Lors de la création d'une table (column family), on doit définir ces 3 types qui ne peuvent plus être modifiés par la suite.

```
create column family user
 with key_validation_class = LongType
 and comparator = UTF8Type
 and default_validation_class = UTF8Type
```

- key\_validation\_class correspond au type de la #partition
- comparator correspond au type de la #col
- default\_validation\_class correspond au type de la cellule

Voici un contenu possible de la table user en utilisant le client cassandra-cli avec l'API Thrift :

```
[default@test] list user;
Using default limit of 100
Using default cell limit of 100
```

```

RowKey: 10
=> (name=age, value=32, timestamp=1403441029826000)
=> (name=nom, value=MARTIN, timestamp=1403441024052000)
```



## NFE204 – Bases de données documentaires et distribuées

```
=> (name=prenom, value=Jean, timestamp=1403441027825000)
```

```

```

```
RowKey: 11
```

```
=> (name=age, value=26, timestamp=1403441047501000)
```

```
=> (name=nom, value=DUCROS, timestamp=1403441034463000)
```

```
⇒ (name=prenom, value=Elise, timestamp=1403441039578000)
```

```
⇒ RowKey correspond à #partition.
```

### ABSTRACTION

D'une manière générale, une table dans C\* peut être assimilée à une structure de données.

Map<#partition, SortedMap<#col, cellule>>

On retrouve le tri des #col avec la SortedMap (Map triée). La Map externe n'est pas triée si on utilise un partitionnement aléatoire qui disperse les #partition sur tous les noeuds.

Si on avait choisi un partitionnement ordonné, la structure d'une table serait assimilable à :

SortedMap<#partition, SortedMap<#col, cellule>>

## 4.5 CASSANDRA ET LES SÉRIES TEMPORELLES

---

Cassandra dispose d'un mécanisme propre permettant de trier les données avant leurs écritures séquentielles sur le disque. Pour la lecture de ces mêmes données, un mécanisme d'accès rapide et efficace facilite (critères : clé et rang) la récupération de ces données.

➔ Ce mécanisme se veut totalement adapté aux séries temporelles.

Cassandra étend la possibilité des SGBD NoSQL classique en ce qui concerne les séries temporelles : il est possible de développer des modèles de données de séries temporelles garantissant la performance pour la lecture et l'écriture en base.

Étudions quelques exemples de modèles :

Les cas décrits ci-dessous présentés par la société Datastax, permettent de mieux comprendre le fonctionnement de Cassandra :

Ci-dessous, Les données en entrées sont générées toutes les secondes par une station météo

# NFE204 – Bases de données documentaires et distribuées

## 1<sup>er</sup> pattern : Stockage en colonnes dynamiques

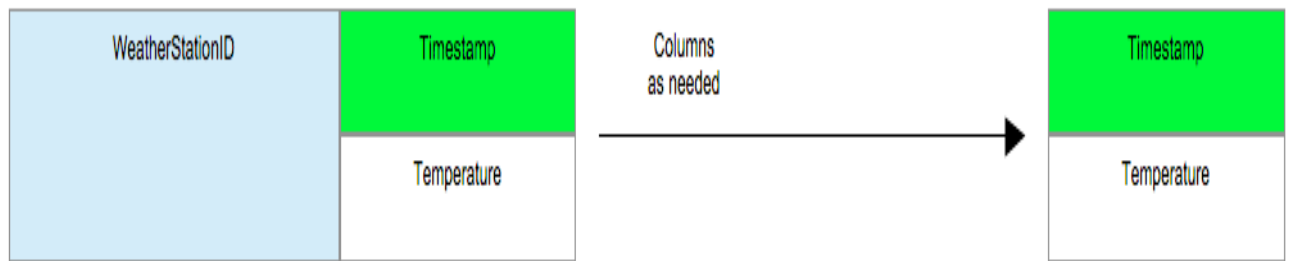


Figure 1

Au fur et à mesure que les données sont créées, celles-ci sont timestampées et insérées dynamiquement dans une colonne. La ligne (Rangée) va s'agrandir au fur et à mesure que les données sont insérées. Une rangée est définie par une clé (WeatherStationID)

Au niveau CQL,

Requête de création de la table :

```
CREATE TABLE temperature (weatherstation_id text, event_time timestamp, temperature text, PRIMARY KEY (weatherstation_id,event_time));
```

Requêtes d'insertion des données :

```
INSERT INTO temperature(weatherstation_id,event_time,temperature) VALUES ('1234ABCD','2013-04-03 07:01:00','72F'); INSERT INTO temperature(weatherstation_id,event_time,temperature) VALUES ('1234ABCD','2013-04-03 07:02:00','73F'); INSERT INTO temperature(weatherstation_id,event_time,temperature) VALUES ('1234ABCD','2013-04-03 07:03:00','73F'); INSERT INTO temperature(weatherstation_id,event_time,temperature) VALUES ('1234ABCD','2013-04-03 07:04:00','74F');
```

Requête de lecture des données :

```
SELECT event_time,temperature FROM temperature WHERE weatherstation_id='1234ABCD';
```

Cassandra ordonne les données lors de leur écriture sur disque, cette ordonnancement selon le timestamp permet de facilement requêter la base et ainsi obtenir une réponse rapide et adaptée.

## 2ème pattern : Partitionnement logique des données

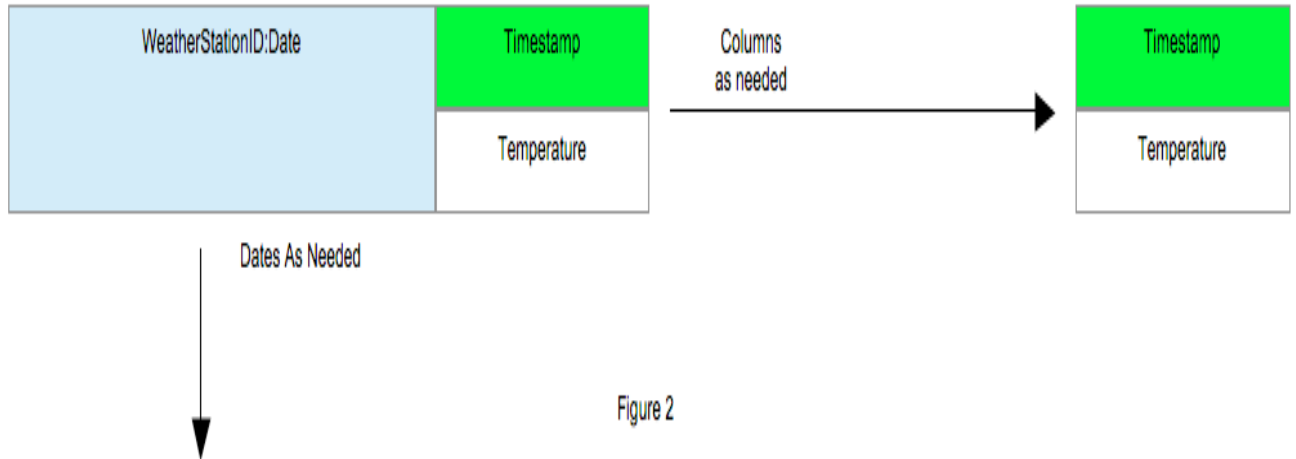


Figure 2

Selon la typologie de données, une rangée peut vite arriver à saturation. Cassandra peut insérer jusqu'à 2 milliards de colonnes par rangée. Il est conseillé de ne pas risquer d'atteindre cette limite.

Pour se faire, Cassandra propose, à la manière du SGBD classique, la possibilité de transformer la clé unique vue dans le 1er pattern en une clé composée construite en partie à partir du timestamp.

La clé composée pour notre exemple sera constituée de weatherstation\_id et d'un champ date.

Au niveau CQL,

Requête de création de la table :

```
CREATE TABLE temperature_by_day (weatherstation_id text, date text, event_time timestamp, temperature text,
PRIMARY KEY ((weatherstation_id,date),event_time));
```

Requêtes d'insertion des données :

```
INSERT INTO temperature_by_day(weatherstation_id,date,event_time,temperature) VALUES
('1234ABCD','2013-04-03','2013-04-03 07:01:00','72F'); INSERT INTO
temperature_by_day(weatherstation_id,date,event_time,temperature) VALUES ('1234ABCD','2013-04-
03','2013-04-03 07:02:00','73F'); INSERT INTO
temperature_by_day(weatherstation_id,date,event_time,temperature) VALUES ('1234ABCD','2013-04-
04','2013-04-04 07:01:00','73F'); INSERT INTO
temperature_by_day(weatherstation_id,date,event_time,temperature) VALUES ('1234ABCD','2013-04-
04','2013-04-04 07:02:00','74F');
```

Requête de lecture des données :

```
SELECT * FROM temperature_by_day WHERE weatherstation_id="1234ABCD" AND date="2013-04-03";
```

Un tel pattern permet de partitionner les données selon une date et offre une souplesse pour requêter selon la clé définie.

# NFE204 – Bases de données documentaires et distribuées

## 3ème pattern : Nettoyage des données

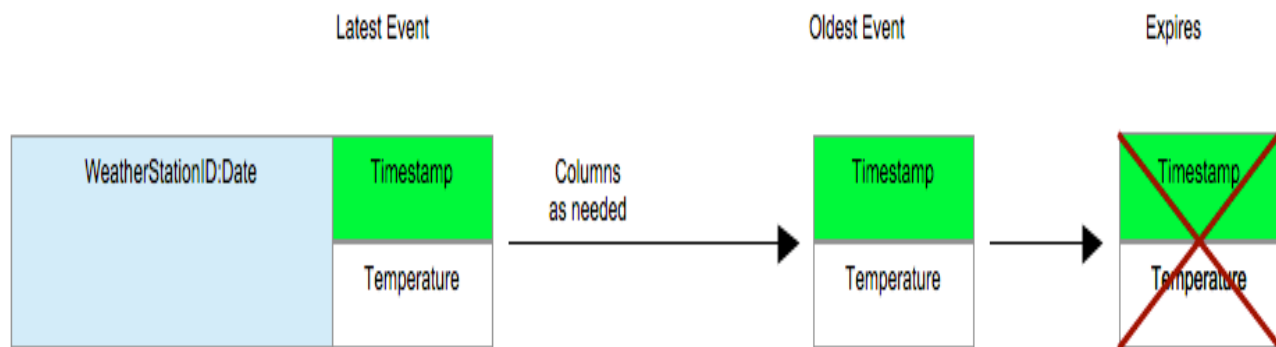


Figure 3

Cassandra propose la faculté de nettoyer les données insérées au bout d'un certain temps. C'est un procédé efficace quand on veut s'éviter la manipulation de données devenues inutiles.

A l'insertion c'est la commande 'USING TTL' qui nous permet de générer cela.

Au niveau CQL,

Requête de création de la table :

```
CREATE TABLE latest_temperatures (weatherstation_id text, event_time timestamp, temperature text,
PRIMARY KEY (weatherstation_id,event_time),) WITH CLUSTERING ORDER BY (event_time DESC);
```

Requêtes d'insertion des données :

```
INSERT INTO latest_temperatures(weatherstation_id,event_time,temperature) VALUES ('1234ABCD','2013-04-03 07:03:00','72F') USING TTL 20; INSERT INTO
latest_temperatures(weatherstation_id,event_time,temperature) VALUES ('1234ABCD','2013-04-03 07:02:00','73F') USING TTL 20; INSERT INTO latest_temperatures(weatherstation_id,event_time,temperature)
VALUES ('1234ABCD','2013-04-03 07:01:00','73F') USING TTL 20; INSERT INTO
latest_temperatures(weatherstation_id,event_time,temperature) VALUES ('1234ABCD','2013-04-03 07:04:00','74F') USING TTL 20;
```

Dans le cas suivant, les données insérées sont supprimées toutes les 20 secondes.

**On peut le dire, Cassandra offre une aisance de manipulation comparée à MongoDB dans le cas des séries temporelles. Simplement, on peut regretter le manque de souplesse dans la mise en forme d'un modèle de données : en effet on peut vite se rendre compte qu'un modèle développé au préalable peut s'avérer figé avec le temps. Disons que Cassandra nécessite de bien penser aux tables et aux clés à créer pour pouvoir requêter facilement. MongoDB quant à lui permet de « jouer » avec le modèle si celui-ci ne convient plus.**

## 1.1 PERFORMANCES

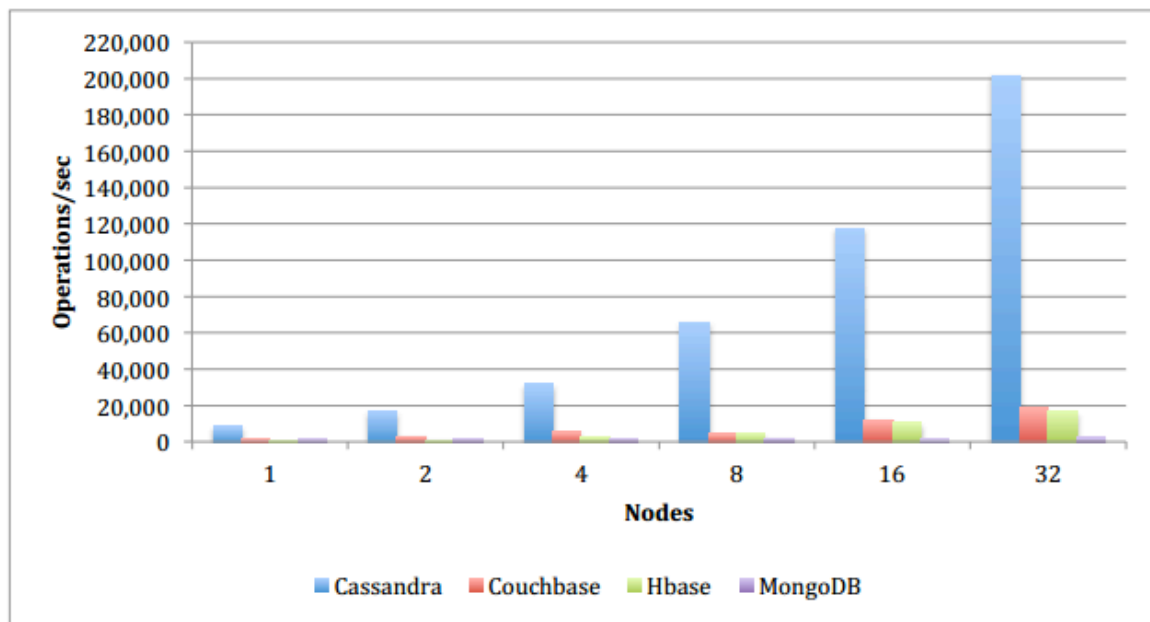
Les utilisateurs l'affirment : MongoDB ne tient pas la cadence en termes de performance comparées à Cassandra dans le cas de grandes volumétries. Sur le long terme, les performances de MongoDB vont

## NFE204 – Bases de données documentaires et distribuées

en décroissance malgré l'emploi de mécanisme de partitionnement physique « Sharding » permettant de répartir au mieux la charge sur plusieurs nœuds de stockage alors que Cassandra se montre plus stable.

Une étude réalisée par Datastax ([http://www.datastax.com/wp-content/themes/datastax-2014-08/files/NoSQL\\_Benchmarks\\_EndPoint.pdf](http://www.datastax.com/wp-content/themes/datastax-2014-08/files/NoSQL_Benchmarks_EndPoint.pdf)), nous montre à quel point des systèmes NoSQL peuvent répondre différemment aux exigences de performance : La figure suivante donne des indications chiffrées sur la charge de travail permises sur les systèmes NoSQL (Cassandra, CouchBase, Hbase, MongoDB) en mode lecture-MAJ-écriture.

### Read-Modify-Write Workload



| Nodes | Cassandra  | Couchbase | HBase     | MongoDB  |
|-------|------------|-----------|-----------|----------|
| 1     | 8,578.84   | 1,326.77  | 324.8     | 1,261.94 |
| 2     | 16,990.69  | 2,928.64  | 961.01    | 1,480.72 |
| 4     | 32,005.14  | 5,594.92  | 2,749.35  | 1,754.30 |
| 8     | 65,822.21  | 4,576.17  | 4,582.67  | 2,028.06 |
| 16    | 117,375.48 | 11,889.10 | 10,259.63 | 1,114.13 |
| 32    | 201,440.03 | 18,692.60 | 16,739.51 | 2,263.69 |

L'étude n'indique pas quel modèle de données a été implémenté dans chaque système mais nous pouvons reconnaître que les statistiques affichées sont en largement en faveur de Cassandra.